

华中科技大学光学与电子信息学院
《信号与线性系统》课程

工程设计问题设计报告

题目：语音伪装器设计

姓 名：陈昀泽

学 号：U202413823

时 间：2025.12.1

指导教师：程孟凡

目录

1	问题描述	1
1.1	原始话题情境	1
1.2	问题扩展与定义	2
1.3	建模要点和评估指标	2
2	理论模型	3
2.1	原理分析与设计思路	3
2.2	数学模型	4
3	程序设计	5
3.1	编程思路	5
3.2	主要流程图及说明	7
3.2.1	模块说明	8
3.3	结果分析	9
3.3.1	运行结果展示	9
3.3.2	结果验证	12
3.3.3	技术指标	13
4	AI 协同设计分析	16
5	总结与体会	17
A	MATLAB 代码实现	20
A.1	主程序文件	20
A.2	核心算法实现	22
A.2.1	相位声码器时间拉伸	22
A.2.2	STFT 频谱插值	23
A.2.3	完整变调功能	24
A.2.4	共振峰校正方法	25
A.3	分析与可视化工具	30
A.3.1	共振峰对比分析	30
A.3.2	LPC 共振峰估计	34
A.4	测试与演示程序	35
A.4.1	变调共振峰分析演示	35
A.4.2	倒谱域校正演示	41
B	代码使用说明	43

摘要

本文面向隐私保护与人机交互等典型应用场景，设计并实现了一套能够在男声、女声与童声之间进行转换的语音变音系统。系统以源—滤波器模型为理论基础，采用短时傅里叶变换进行时频分析，并以相位声码器（PVOC）实现时间—音高解耦，在不破坏相位一致性的前提下通过重采样完成基频变换，能够改变声音的音调高低；在完成题目基础任务之外，为缓解仅变调时产生的“芯片人效应”，除了改变基频外，还需要改变共振峰来改变声道特性。本文实现了一个倒谱域共振峰保持算法，控制共振峰整体迁移，能够改变决定听感的声道特性。开发过程中结合 AI 助手开展资料检索、方案对比与参数区间设定，确定以相位声码器为基础，扩展倒谱包络伸缩为核心的实现路径。综合结果表明，本文方法在相同音高变换条件下引入可控参数 以实现音色迁移，在主观听感上更贴近目标性别/年龄感的音色迁移与更高的主观自然度，对比 Matlab 内置的 shiftPitch，在变声自然度上显著优于 shiftPitch 的共振峰迭代校正算法。但在包络贴合的客观指标（如 MSE、MAE）上与 Matlab 内置的 shiftPitch 仍存在明显差距。整体而言，本文从零复现了相位声码器，并且提出了一种倒谱域共振峰保持算法，在满足课程指标的同时，基于共振峰缩放这一扩展模块实现相对于基线更自然的音色转换，为课程场景下的从零复现与扩展研究提供了可解释、可调参的平台。

关键词：语音伪装、相位声码器、基频变换、共振峰缩放、倒谱分析

1. 问题描述

1.1 原始话题情境

在隐私保护、安防通讯和娱乐交互等实际场景中，常需要对语音进行主动变换，以隐藏说话人的身份特征。人的声音差异主要由音调（基频 F_0 ）与音色（由声道结构产生的共振峰 Formants）共同决定。通过同时调整基频与共振峰分布，可在男声、女声与童声等类别之间实现听感上的转换。本工程任务遵循课程题目要求，利用短时傅里叶变换（STFT）、滤波器设计与频率映射等工具，设计一个可实时或准实时的语音伪装器，使输出在满足目标听感的同时保持自然度与可懂度，并支持慢录快放与快录慢放等功能演示。

1.2 问题扩展与定义

从“信号与线性系统”的角度，可将语音产生建模为源-滤波器（Source-Filter）模型。语音信号 $s(t)$ 由激励源 $e(t)$ 经代表声道的线性时变系统 $h(t)$ 产生：

$$s(t) = (e * h)(t), \quad S(\omega) = E(\omega)H(\omega),$$

其中， $E(\omega)$ 包含基频与谐波信息， $H(\omega)$ 的谱峰构成共振峰，决定音色特征。语音伪装器的目标是设计处理系统 T （或其频域映射），使输出在基频与共振峰方面达到预期的感知变化。为统一表述，定义两类缩放参数：基频缩放系数 α 与共振峰缩放系数 β ，分别满足

$$F'_0 = \alpha F_0, \quad F'_k = \beta F_k,$$

其中 $\alpha = 2^{\text{semitones}/12}$ 控制升降调（ $\alpha > 1$ 升调， $\alpha < 1$ 降调）， β 控制共振峰整体伸缩（ $\beta > 1$ 上移、更尖亮； $\beta < 1$ 下移、更厚重）。因此，语音伪装可视为“频谱重塑（spectral reshaping）”问题：在保持语义内容的前提下，分别对激励谱与声道包络实施可控映射，使输出频谱特性逼近目标类别的理想结构。为达成该目标，需在短时稳态假设下将基频信息与共振峰包络信息分离并独立处理。本文采用 STFT 分析与相位声码器（PVOC）进行时间-音高解耦，以时间伸缩配合重采样改变基频；在此基础上，对频谱包络实施频率轴伸缩，实现对共振峰的调整/缩放，从而实现 α 与 β 的解耦控制。

1.3 建模要点和评估指标

语音信号为典型非平稳过程，短时内近似平稳，故采用 STFT 获取时频分布，作为基频估计、共振峰检测与频谱重塑的基础。记帧索引 m 、频率索引 k ，窗函数 $w[n]$ 与 FFT 点数 N ，则

$$X[m, k] = \sum_n x[n] w[n - mR] e^{-j2\pi kn/N},$$

其中 R 为帧移。基频变换可视为对激励谱的频率伸缩：设半音数为 semitones，则 $\alpha = 2^{\text{semitones}/12}$ ，相当于设计音高变换使 $E'(\omega) \approx E(\omega/\alpha)$ ；为避免直接时域拉伸带来的相位失真，采用相位声码器以保证相位一致性。共振峰变化对应于对声道频谱包络的非线性频率轴扭曲，可建模为包络变换算子 G_β ：

$$\text{Env}_{\text{out}}(\omega) = \text{Env}_{\text{in}}(\omega/\beta),$$

其参数 β 反映目标音色类别的共振峰整体相对位置（例如女声相较男声具有更高的共振峰中心频率）。实际实现可采用两类方法：倒谱分析在倒谱域通过低通升析窗（lifter）分离包络并对频率轴进行伸缩；LPC 系统模型通过极点映射实现共振峰搬移。两者均可形成可控的“音色调节模块”，但后续评估显示，LPC 系统模型的效果较差，反而会破坏共振峰的原始特性。综合上述两部分，整体映射实现对 $E(\omega)$ 与 $H(\omega)$ 的分离控制，使语音伪装成为可分析、可设计的频域系统工程问题。

在评价指标方面，本文采用包络贴合度量为主：为量化共振峰调整的有效性，分别对未进行 β 的基线，倒谱域共振峰伸缩法（ β 控制）、LPC 共振峰伸缩法以及 shiftPitch（Matlab 中成熟的变调及校正函数）可视化评价，评价对象为输出包络 Env_Y 与目标包络 Env_{target} 的接近程度；分别对不同情况下的频谱进行 MSE、MAE、Correlation、SD 四类指标计算，评价其与原信号的贴合度、形状一致性和稳定性，反映输出信号频谱包络与原信号在关键频带内的匹配质量。

2. 理论模型

2.1 原理分析与设计思路

语音伪装的核心目标是在维持原始语义内容完整性的前提下，通过解耦地改变语音信号的基频与共振峰分布，从而重塑说话人的音调与音色，使听者感知到目标类别（男声、女声、童声等）的听觉特征。依据源-滤波器模型，自然语音信号可被视为“激励源—声道系统”框架下的产物：声带振动产生的准周期激励决定基频特性，声道系统的滤波作用形成反映音色的共振峰包络。基于该物理模型，语音伪装可分解为两部分的独立控制：其一，改变激励源的时间尺度与采样率以实现基频的线性伸缩；其二，对声道系统的频谱包络实施频率轴伸缩以调整共振峰的整体位置分布。为实现上述解耦控制，本文采用“PVOC 时间尺度变换 + 重采样的音高伸缩 + 倒谱包络伸缩”的级联策略：首先，使用相位声码器（PVOC）在 STFT 域对时频帧进行重组，通过相位预测与解缠保证水平与垂直相位一致性，实现对时长的无失真拉伸/压缩；其次，通过重采样改变采样率，使频谱在频率轴上按 $\alpha = 2^{\text{semitones}/12}$ 进行线性伸缩并将时长恢复至原始长度，从而完成音高变换；最后，在倒谱域估计平滑的对数频谱包络，对其实施频率轴伸缩（由参数 β 控制），构造目标包络并以对数谱比值的形式对变调后的谱进行加权，使输出的共振峰分布逼近目标类别。这一设计将复杂的非线性音色变换转化为可分离、可分析的频域重塑过程，同时避免仅变调时出现的“芯片人效应”，并为课程要求中的慢录快放/快

录慢放与滤波器设计提供统一的时频分析框架。

2.2 数学模型

- (1) 语音产生模型与频域表示在源-滤波器 (Source-Filter) 模型下, 语音信号 $s(t)$ 由激励源 $e(t)$ 经声道系统 $h(t)$ 产生:

$$s(t) = (e * h)(t), \quad S(\omega) = E(\omega) H(\omega),$$

其中 $E(\omega)$ 包含基频 F_0 及其谐波结构, $H(\omega)$ 的谱峰构成共振峰 F_k 并决定音色。为实现语音伪装, 本文引入双参数缩放:

$$F'_0 = \alpha F_0, \quad F'_k = \beta F_k,$$

其中 $\alpha = 2^{\text{semitones}/12}$ 控制升降调, β 控制共振峰整体伸缩 (近似声道长度归一化的效果)。

- (2) 短时傅里叶变换 (STFT) 与时频表示对输入信号 $x[n]$ 进行分帧加窗, 窗函数采用 Hann 窗 $w[n]$, 帧移为 R , FFT 点数为 N , 其 STFT 表示为

$$X[m, k] = \sum_n x[n] w[n - mR] e^{-j2\pi kn/N},$$

STFT 矩阵为后续 PVOC 的时间尺度变换、基频估计与包络提取提供时频基础。

- (3) 基于相位声码器的时间尺度修改 (TSM) 设时间尺度变换因子为 Q ($Q > 1$ 为拉伸, $Q < 1$ 为压缩)。PVOC 通过分析相邻帧的相位差并进行相位解缠与预测, 构造新的合成网格, 实现相位一致的时间重排。记第 m 帧第 k 频点的分析相位为 $\phi_a[m, k]$, 则去除期望相位推进后的瞬时频率偏差 $\Delta\omega[m, k]$ 用于合成相位的递推:

$$\phi_s[m, k] = \phi_s[m - 1, k] + \Delta\omega[m, k] \frac{R_s}{N},$$

其中 R_s 为合成帧移, 满足 $R_s = Q R$ 。该过程在不改变频率成分位置的前提下实现时长的伸缩, 并保持水平相位连续性。

(4) 音高变换模型：TSM + 重采样设目标半音数为 semitones，音高缩放因子为

$$\alpha = 2^{\text{semitones}/12}.$$

算法策略为：先用 PVOC 将时长变为原来的 Q 倍（基频位置保持），再对 PVOC 输出执行重采样，使采样率按 α 缩放并将时长恢复至原始，得到

$$y_{\text{pitch}}[n] \Rightarrow Y_{\text{pitch}}(\omega) \approx X\left(\frac{\omega}{\alpha}\right),$$

此时频谱各成分（含 F_0 与谐波）完成线性伸缩，实现音高变换且时长回到原始。

(5) 共振峰调整模型：倒谱包络伸缩与比值滤波重采样后，共振峰将随谱整体伸缩产生偏移。为将音色调整至目标类别，本文在倒谱域估计平滑的对数频谱包络 $\text{Env}(\omega)$ ，并实施频率轴伸缩：

$$\text{Env}_{\text{target}}(\omega) = \text{Env}_{\text{orig}}\left(\frac{\omega}{\beta}\right),$$

其中 $\beta > 1$ 对应共振峰整体上移， $\beta < 1$ 对应整体下移。令 $Y(\omega)$ 为变调后帧的复谱，对其包络估计为 $\text{Env}_Y(\omega)$ ，则采用对数谱比值构造校正增益

$$G(\omega) = \exp\left(\text{Env}_{\text{target}}(\omega) - \text{Env}_Y(\omega)\right),$$

并对谱进行加权得到

$$Y_{\text{adj}}(\omega) = Y(\omega) \cdot G(\omega),$$

最后经逆变换与重叠相加（OLA）合成到时域，获得包络已按 β 调整的输出语音 $y[n]$ 。该包络伸缩等价于在频率轴上对声道系统响应实施线性扭曲（VTLN 式），从而实现对共振峰的显式调整/缩放。

3. 程序设计

3.1 编程思路

本系统旨在实现语音信号的变调与共振峰调整处理，其核心基于相位声码器（Phase Vocoder, PVOC）的时频分析框架，结合重采样技术实现基频调整，并通过倒谱同态方法实现共振峰包络的频率轴缩放。整体逻辑采用“先时间伸缩，后重采样，再包络缩放”的级联策略，确保基频缩放（由 α 控制）与共振峰缩放（由 β 控制）解耦。

(1) 变调主流程设计

输入音频首先经过 STFT 模块进行分帧加窗处理，获取其时频域表示。系统根据用户设定的目标半音变化量 n 计算音高缩放因子 $\alpha = 2^{n/12}$ ，选取时间尺度因子 Q （工程上取 $Q \approx 1/\alpha$ 以便恢复时长），并调用核心模块 PVOC 将信号的时间轴拉伸或压缩为原来的 Q 倍。在此基础上，对 PVOC 输出执行重采样操作，采样率变化因子设定为 α ，这一步在将时长恢复至原始长度的同时，使频谱在频率轴上发生线性伸缩，实现目标的音高变化。随后在倒谱域对频谱包络实施频率轴伸缩，按参数 β 构造目标包络并进行频域比值加权，实现共振峰的显式调整/缩放。

(2) 相位声码器实现

相位声码器模块实现遵循经典 PVOC 结构，并针对语音信号特性进行参数优化。为精细捕获相位演变并减少合成伪影，分析步长设为帧长的四分之一（约 75% 重叠）。在 pvsample 子程序中，算法通过计算相邻帧相位差，并扣除由跳步产生的期望相位推进，得到瞬时频率偏差；在合成阶段于新的时间网格上进行相位累积，保持水平相位连续性，从而在改变时长 Q 的同时尽可能避免相位失真引起的“金属感”。与现成库函数不同，本实现从零搭建了 STFT 分析、相位累积与幅度插值等关键环节，便于复现实验细节。

(3) 共振峰调整模块设计

变调（PVOC+ 重采样）会引起共振峰位置的整体漂移。为实现与目标音色类别一致的共振峰分布，系统采用倒谱同态方法在对数谱域估计平滑包络，并以频率轴伸缩的方式进行调整。具体做法为：分别从原始信号与变调后信号估计平滑对数包络 $\text{Env}_{\text{orig}}(\omega)$ 与 $\text{Env}_Y(\omega)$ ，据此定义目标包络

$$\text{Env}_{\text{target}}(\omega) = \text{Env}_{\text{orig}}\left(\frac{\omega}{\beta}\right),$$

其中 $\beta > 1$ 表示共振峰整体上移、 $\beta < 1$ 表示整体下移。随后构造频域增益

$$G(\omega) = \exp(\text{Env}_{\text{target}}(\omega) - \text{Env}_Y(\omega)),$$

并对变调后谱进行加权，实现对共振峰的显式缩放，使输出包络贴合 β 控制的目标分布。

数值稳定性与参数策略在对数谱计算中引入幅度下限以抑制静音段导致的数值发散；STFT 与 ISTFT 均采用 Hann 窗并配合重叠相加（OLA）进行增益归一化，消除窗引入的幅度调制；对 $G(\omega)$ 可做适度平滑与限幅以提升稳健性。综合上述策略，底层实现形成“PVOC 时间伸缩 + 重采样变调 + 倒谱包络缩放”的可控语音伪装流程。

3.2 主要流程图及说明

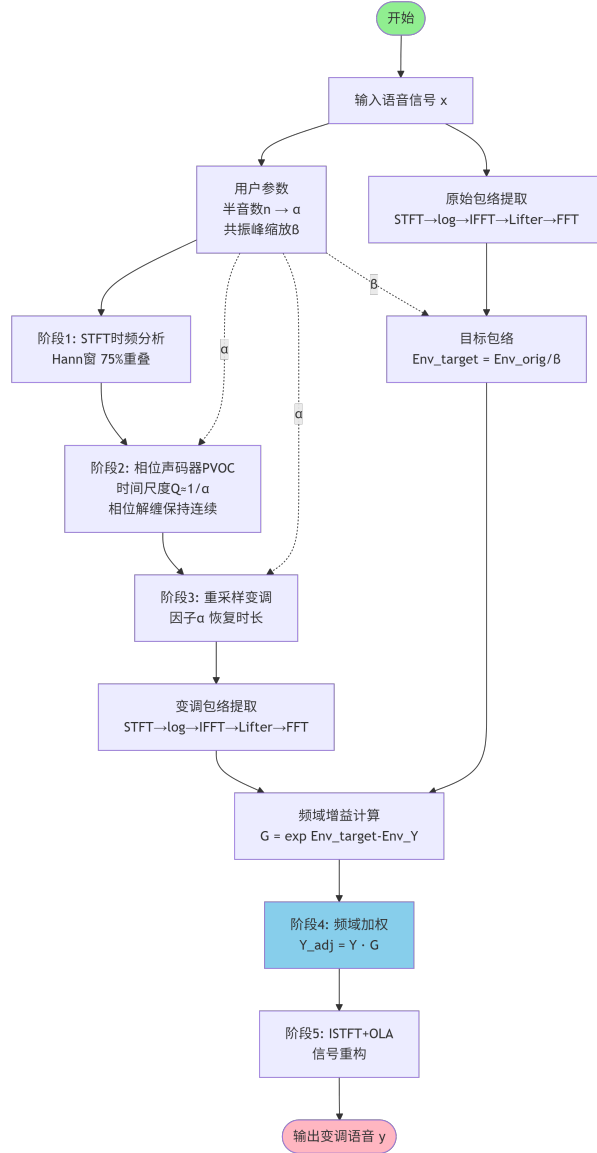


图 1: 系统总体流程图

3.2.1 模块说明

阶段一：时频分析（Time-Frequency Analysis）

STFT（短时傅里叶变换）：将一维时域语音 $x[n]$ 转换为二维时频矩阵 $X[m, k]$ 。系统采用 Hann 窗以降低频谱泄漏，并设定约 75% 的帧重叠率（Hop Size = Frame Size / 4），为后续高精度相位预测与无痕重构奠定基础。

阶段二：时间尺度修改（Time-Scale Modification, TSM）

相位声码器（PVOC）：作为变调的前置步骤。模块接收由半音数 n 导出的时间尺度 Q （工程上 $Q \approx 1/\alpha$ ），在频域内对信号进行时间轴拉伸/压缩。其核心通过瞬时频率估计与相位累积，在改变信号时长 Q 的同时保持水平相位连续，避免传统插值带来的

瞬态失真。

阶段三：重采样变调（Resampling-based Pitch Shifting）

重采样：该模块执行音高伸缩。将经过 TSM 的信号（时长为原始 Q 倍）按 α 进行重采样，使频谱在频率轴上整体线性伸缩并将时长恢复至原始，从而实现目标的音高变化 $F'_0 = \alpha F_0$ 。

阶段四：倒谱共振峰缩放（Cepstral Formant Scaling）

倒谱域共振峰缩放：分别计算原始与变调信号的对数谱并取实倒谱，应用低通升析窗分离平滑包络。目标包络定义为 $\text{Env}_{\text{target}}(\omega) = \text{Env}_{\text{orig}}(\omega/\beta)$ ，并据此构造频域增益 $G(\omega) = \exp(\text{Env}_{\text{target}} - \text{Env}_Y)$ ，作用于变调后频谱以实现共振峰的整体伸缩与搬移。

阶段五：信号重构（Signal Reconstruction）

ISTFT（逆短时傅里叶变换）：将经包络缩放后的复数频谱转换回时域，并通过重叠相加（OLA）实现幅度增益归一化与帧间连续性，最终得到高保真度的输出 $y[n]$ ，其基频与共振峰分别由 α 与 β 解耦控制。

3.3 结果分析

3.3.1 运行结果展示

在 Matlab 环境下，系统完成了语音录制、频谱分析、数字滤波降噪、基频变换（由 $\alpha = 2^{n/12}$ 控制）、共振峰缩放（由 β 控制）、以及慢录快放/快录慢放等模块。界面可视化实时展示时域波形、幅度谱/包络与基频轨迹，便于对比不同参数设置的效果。处理流程为：先通过 PVOC 实现时间尺度修改（ Q ），再经重采样完成音高伸缩并恢复时长（ α ），最终在倒谱域对平滑包络实施频率轴伸缩，使输出包络逼近 $\text{Env}_{\text{target}}(\omega) = \text{Env}_{\text{orig}}(\omega/\beta)$ 。不同模式（如男 → 女升调、女 → 童升调、女 → 男降调）通过设定 α, β 获得相应的音高与音色转换。整体运行稳定，时频重构平滑自然。

表 1: 常见变声方向参数对照表

变声方向	semitones	β
女声 → 男声	-5	0.75
女声 → 童声	+3	1.20
男声 → 女声	+2.5	1.15
童声 → 女声	-2.8	0.85

图2给出了系统图形界面与典型可视化输出，包括时域波形、频谱/包络与时频图

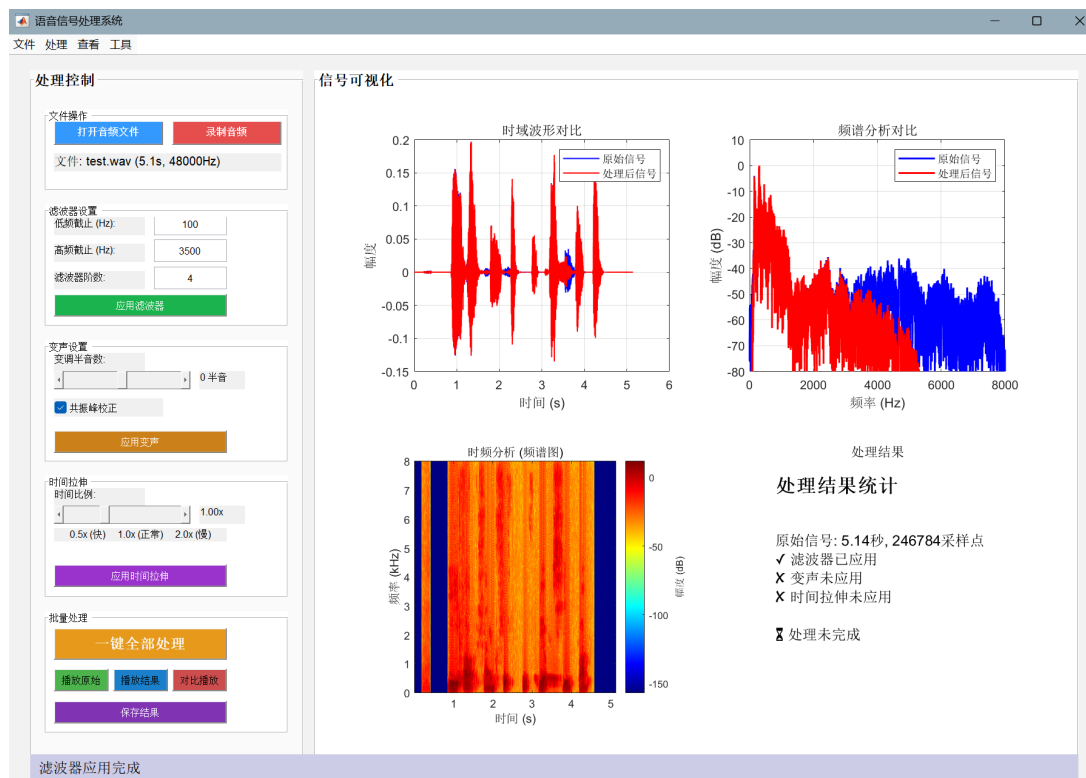


图 2: 系统 GUI 界面与可视化示例

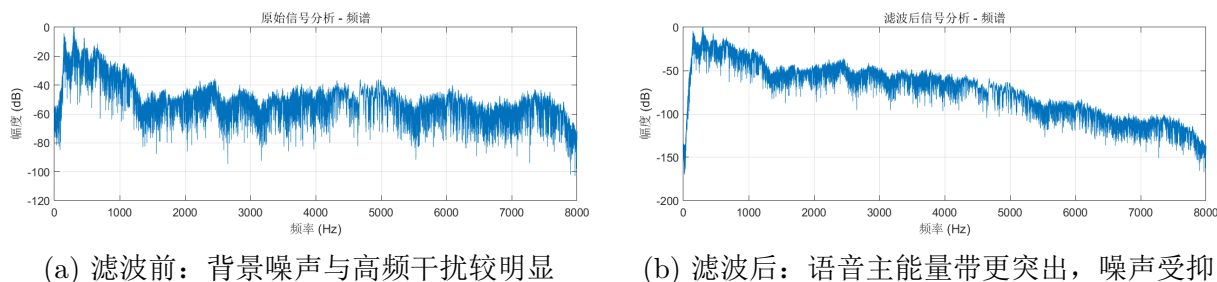
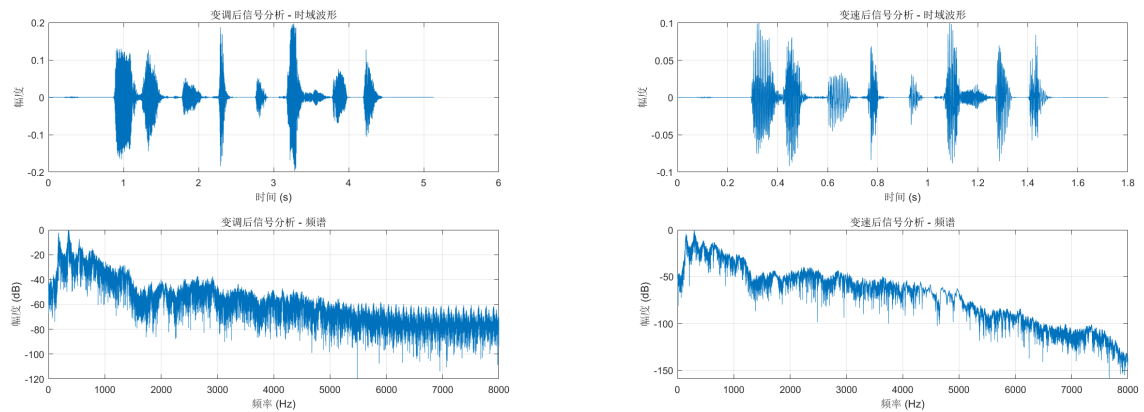


图 3: 带通滤波预处理效果对比（频域视角）

等，能够较为方便快捷地对信号进行测试，并且能够输出滤波/变调/变速前后信号的时域/频域对比，较清晰地展示了信号变换的过程。图3对比了带通滤波前后的频谱，展示了预处理对低频噪声与高频干扰的抑制；图4分别给出了变调结果与变速结果；图5给出了从输入到各处理阶段的总体对比展示，用于直观呈现系统工作流程在不同模块作用下的听感与可视化变化。上述结果与前文数学模型一致：变调通过 $\alpha = 2^{n/12}$ 实现频谱的线性伸缩并恢复时长，变速通过 Q 实现时长的拉伸/压缩且保持谱形一致性；在此基础上，系统进一步在倒谱域实现包络目标缩放（由 β 控制），用于音色迁移（该部分在后续详细量化与讨论）。



(a) 变调结果：频谱整体按比例伸缩且时长恢复

(b) 变速结果：时长改变而谱形基本保持

图 4: 变调与变速效果

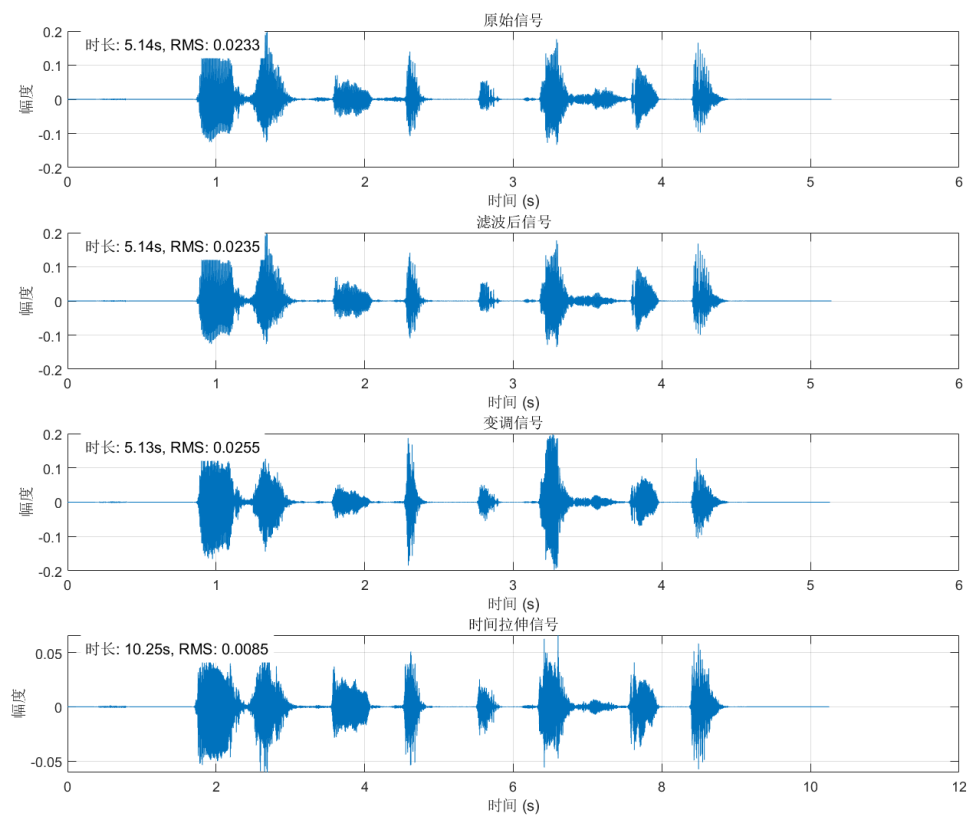


图 5: 总体对比视图

3.3.2 结果验证

本系统的多项功能通过频域与包络域对比得到验证，验证口径与数学模型一致：音高伸缩以 $\alpha = 2^{n/12}$ 为参照，包络伸缩以 $\text{Env}_{\text{target}}(\omega) = \text{Env}_{\text{orig}}(\omega/\beta)$ 为参照。

1. 变声效果验证

(a) 基频搬移验证

在男 $\rightarrow$ 女或女 $\rightarrow$ 童的升调模式下，基频及相关谐波成分整体上移且幅度结构保持稳定；在女 $\rightarrow$ 男或童 $\rightarrow$ 女的降调模式下，基频整体下移。图示对比表明，输出频谱对 α 的线性伸缩满足预期，体现时间—音高解耦流程的有效性。

(b) 共振峰调整验证

仅进行音高变换（仅 α ）时，共振峰位置会随谱整体伸缩而偏离目标音色分布共振峰包络 $\text{Env}_{\text{target}}$ ，并以对数谱比值加权实现共振峰的显式调整/缩放。变调前后图6对比显示，倒谱包络伸缩法得到的共振峰轨迹（绿色点）更接近目标共振峰包络（黑色点），优于 Matlab 中的 `shiftPitch`（紫色点），说明。

(c) 包络频域验证

为展示变声前后频域结构的变化，绘制平滑对数包络的 Bode 图，重点关注 500-3000 Hz 主能量区间。结果显示，采用 β 控制的倒谱包络伸缩后，输出包络在关键频段更贴近 $\text{Env}_{\text{target}}$ ，验证了共振峰调整的有效性。

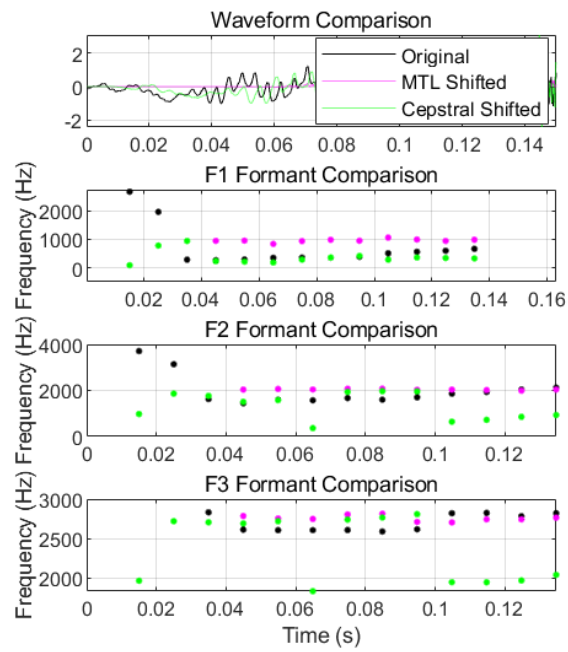


图 6: 共振峰轨迹对比

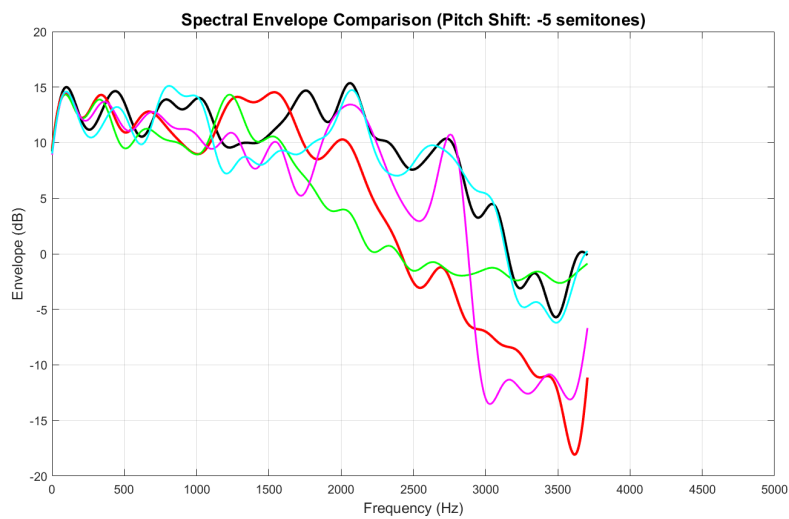
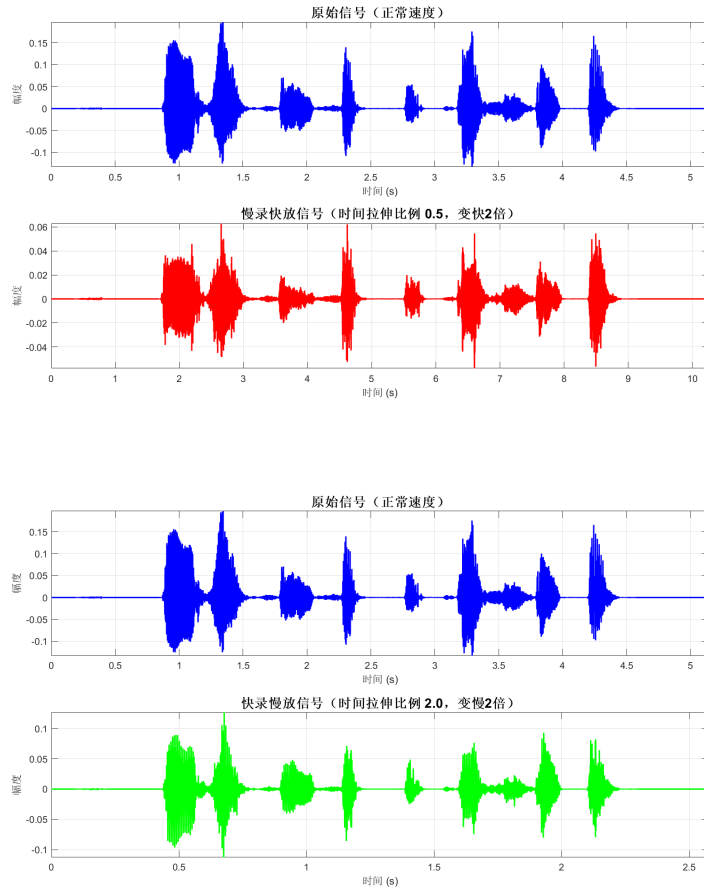


图 7: 频域包络对比图

2. 慢录快放与快录慢放验证

利用 PVOC 的时间尺度修改能力，分别演示 $TSM > 1$ （时间拉伸）与 $TSM < 1$ （时间压缩）两种模式。时域波形图显示，语音时长改变而谱形基本保持，说明 PVOC 的相位预测与重构有效，实现时间尺度与音高伸缩的解耦后变速。



3.3.3 技术指标

为量化包络调整的有效性，分别对倒谱包络伸缩法 (β 控制)、未进行 β 缩放的基线，以及 shiftPitch (Matlab 中成熟的变调及校正函数) 进行 MSE、MAE、Correlation、SD 四类指标计算，评价其与原信号的贴合度、形状一致性和稳定性，反映输出信号频谱包络与原信号在关键频带内的匹配质量。

倒谱域包络缩放相较未调整基线在幅度贴合度上明显提升，形状一致性提高，频带内误差波动减小，说明引入 β 的频率轴伸缩能够有效拉近输出包络与目标包络的距离；然而，与 shiftPitch 相比，当前实现仍存在差距（例如 MSE 3.05、MAE 1.38、Correlation 0.968、SD 1.75），这一方面差异可能与算法目标差异有关：shiftPitch 的核心偏向“共振峰校正”（在保形与稳健性上更强），而倒谱域方法执行的是“共振峰缩放”（主动搬移峰

表 2: 不同变声方法的性能指标对比

指标	未调整共振峰	倒谱域缩放	shiftPitch 校正	LPC 缩放
MSE	50.0163	34.7860	3.0464	67.8432
MAE	5.7169	4.0447	1.3817	5.432
Correlation	0.8803	0.9286	0.9677	0.6753
SD	7.0722	5.8980	1.7454	8.0054

位以实现音色迁移), 在峰位重映射与幅度再分配过程中更容易产生绝对误差; 另一方面也与实现细节相关, 如倒谱阶数与窗长导致的包络过度/欠平滑、静音与弱能量帧的对数谱阈值与增益限幅设置等。综合图示与统计指标, 结论是: 倒谱域缩放在本研究设定的包络指标上优于未调整基线, 说明其包络缩放效果比较好但低于成熟的 shiftPitch; 然而在满足时间—音高解耦的前提下, 控制为音色迁移提供了灵活手段, 能够做到缩放共振峰, 后续将从包络估计平滑度、边界插值与跨帧一致性等方面进一步优化。LPC 缩放的效果最差, 并且指标甚至高于未校正的指标, 故前述分析评估不再考虑 LPC 校正。

综合图示与统计指标, 结果支持本文“时间与音高解耦控制”的设计主张: 在满足音高变换的同时, 通过包络频率轴伸缩实现目标类别的共振峰分布, 使输出语音在自然度与可懂度上更符合预期。

4. AI 协同设计分析

起始阶段，通过初步调研，我原计划直接调用 MATLAB 提供的 ‘shiftPitch’ 函数作为变调模块。然而，功能验证过程中逐步暴露出该黑箱函数的扩展性不足：其内部时频参数（窗函数类型、帧长、重叠率）、相位连续性策略以及对谱包络的处理方式均不可配置，难以满足“仅改变基频、显式控制共振峰分布”的需求，也无法与后续设计的包络细粒度控制模块有效耦合。基于 AI 辅助的技术检索与文档分析，确认 ‘shiftPitch’ 的底层实现属于相位声码器（Phase Vocoder, PVOC）技术路线，因此我们转向“自行复现 PVOC + 重采样 + 包络缩放”的开放式实现方案，便于在分析—合成链路中灵活引入倒谱或线性预测编码（LPC）等包络控制环节，从而实现可解释、可干预、可扩展的模块化系统设计。

在复现 PVOC 过程中，围绕短时傅里叶变换（STFT）、相位展开、相位累积与重叠相加（OLA）重构等关键步骤，出现了多类工程实现与数值稳定性方面的挑战。其中印象最深的是窗函数与重叠区归一化问题：若分析窗与合成窗不匹配，或 OLA 的整体增益未正确归一化，会引入帧间幅度调制和听觉上的“脉冲感”。AI 建议统一采用 Hann 窗，并验证跳跃长度为窗长四分之一（ $\text{hop} = n/4$ ）时所对应的能量补偿系数，从而有效抑制重构信号的能量起伏。

在包络控制方案选择方面，AI 辅助的文献检索与分析表明，单纯改变激励源的基频会导致共振峰整体漂移，造成音色失真，因此需显式调整声道谱包络以保持音色自然。我们围绕“倒谱域共振峰缩放（同态处理）”和“LPC 极点重映射”两条技术路线进行实现与对比：倒谱法通过在对数谱域估计平滑包络 $Env(\omega)$ ，再通过频率轴伸缩实现目标包络分布，其目标包络定义为 $Env_{\text{target}}(\omega) = Env_{\text{orig}}(\omega/\beta)$ ，其中 $\beta > 1$ 表示共振峰上移， $\beta < 1$ 表示下移，增益函数构造为 $G(\omega) = \exp(Env_{\text{target}}(\omega) - Env_Y(\omega))$ ；LPC 方法则通过建立全极点模型来估计声道响应，并对极点对应的角频率进行比例缩放或仿射变换，以实现共振峰的移动。在实现过程中，曾因关键词表述偏差将“倒谱缩放”误写为“倒谱校正”（即默认将包络恢复为原始分布，而非按 β 进行可控缩放），经 AI 定位与建议，迅速将包络比计算公式由 $\exp(env_X - env_Y)$ 修正为 $\exp(env_{\text{target}} - env_Y)$ ，并新增频率轴缩放模块 $env_{\text{target}} = Env_{\text{orig}}(\omega/\beta)$ ，在保持原有倒谱估计与 Lifter 参数不变的前提下，以最小改动实现了共振峰的可控平移。此外，AI 还建议对增益函数 $G(\omega)$ 进行适度平滑与幅值限制，以缓解高 β 值在高频段可能引起的增益尖峰与音色失真；在

LPC 方案中，AI 提示需关注模型阶数选择、预加重处理与稳定性约束（反射系数校验）等关键细节，以降低极点外逸导致的合成失真风险。

在 MATLAB 程序编写过程中，由于本人主要编程经验集中于 Python，对 MATLAB 语法与功能接口不够熟悉，因而频繁出现语法错误与函数使用不当的情况。在此期间，我大量借助 AI 的代码调试与错误修正功能，将报错信息提交给 AI 进行实时分析，显著提升了程序开发效率与问题定位速度。

最终，我们确定系统主体结构为“PVOC 时间伸缩 + 重采样变调 + 倒谱包络缩放”，并与“LPC 极点缩放”方案进行并行对比。该结构的理论依据在于：PVOC 能够在保持频率成分不变的条件下精确调节时间尺度，结合重采样可实现线性音高移动，同时维持相位连续性以降低金属伪影；倒谱同态方法在对数谱域利用低倒频率分量提取平滑包络，具备良好的抗噪性与全局平滑特性，频率轴重映射 $Env_{\text{target}}(\omega) = Env_{\text{orig}}(\omega/\beta)$ 作为业界广泛采用的共振峰移动手段，能够通过单一参数 β 稳定控制音色类型；LPC 则提供了一种机理化的声道建模与参数化调控方式，有利于对比验证与误差溯源。综合主观听感、算法稳健性与实现复杂度，倒谱包络缩放方法在多数实验场景下表现出更平滑的谱包络匹配效果与更低的合成伪影。

通过上述 AI 协同的设计与迭代过程，最终形成了一套结构清晰、参数可控、可解释性强的语音伪装处理技术路线，并在工程实现中显著降低了调试与纠错成本，提升了整体方案的复现性与扩展性。

5. 总结与体会

这次工程设计作业让我把课堂上的信号处理知识真正用到问题上。我学会了先按‘基频’和‘共振峰’拆分任务，再分别选用合适的方法去处理：用 PVOC 和重采样做音高变换，用倒谱做包络伸缩。实践证明，合理的重叠率和相位处理能显著减少金属感，适度的阈值和增益平滑能提升稳定性。相比一开始直接调用黑箱函数，我现在更清楚每个模块在做什么、参数如何影响结果，也更能解释成功与失败背后的原因。与此同步，我也从倒谱与 LPC 两种方法的并行实现中学到了如何在工程层面权衡“包络平滑性、实现复杂度与鲁棒性”，并用客观指标与听感对照来支撑方案选择。

AI 在三个方面帮了我：第一，帮我把黑箱函数的原理摸清楚，让我敢于自己搭建 PVOC 和包络模块；第二，在调试时快速定位问题，很多看似玄学的失真，其实是参数或边界处理不当；第三，在我只改基频导致音色变差时，提供了‘包络伸缩’这条简单

可控的改法。结果上，我的实现比未校正的基线更接近目标包络，但与 shiftPitch 的成熟方案仍有差距。承认差距、找准原因，比盲目“挽尊”更重要。下一步我会重点改进包络估计的平滑与一致性、增益处理的稳健性，并完善主观听感与客观指标的联合评估，把系统做得更稳、更好用。

总体而言，这次收获最大的不是单纯完成了一个功能齐全的系统，而是掌握了把看似复杂的音频处理问题拆解为可操作的信号处理模块的方法论：先明确物理模型与信号结构，再选取合适的数学工具与工程策略，最后用规范的评估指标与可视化验证结果。生成式人工智能在提供知识背景、给出合理的设计备选、协助定位与修复编码问题方面起到了显著的增效作用，但关键的原理理解、参数取舍与结果判读仍然需要我自身的分析与判断。后续工作将围绕实时性优化（低延时 PVOC 与块处理）、主观听感评测体系完善（结合客观包络误差与主观评分）、更丰富的目标音色建模（基于语料统计的共振峰分布）等方向继续推进，使系统在实用性与可解释性方面进一步提升。

参考文献

- [1] MathWorks. *Formant Estimation with LPC Coefficients*. 2024b Documentation. <https://ww2.mathworks.cn/help/releases/R2024b/audio/ref/shiftpitch.html>
- [2] Boersma, P. & Weenink, D. *Praat: doing phonetics by computer*. Version 6.3.09. <http://www.praat.org/>
- [3] Praat Development Team. *praat/praat.github.io*. GitHub repository. Commit 237a309. <https://github.com/praat/praat.github.io/tree/237a309/LPC>
- [4] DeepWiki. *Praat LPC and Formant Analysis*. Last indexed: 13 September 2025. <https://deepwiki.org/praat/praat/wiki/LPC-and-Formant-Analysis>
- [5] Ellis, D. P. W. *A Phase Vocoder in Matlab*. Technical Report, Columbia University, 2002. <http://www.ee.columbia.edu/~dpwe/resources/matlab/pvoc/>
- [6] MathWorks. *Envelope Extraction - MATLAB & Simulink*. 2024b Documentation. <https://ww2.mathworks.cn/help/signal/ug/envelope-extraction.html>

- [7] MathWorks. *Pitch Shifting and Time Dilation Using a Phase Vocoder in MATLAB*. 2024b Documentation. <https://ww2.mathworks.cn/help/audio/ug/pitch-shifting-and-time-dilation-using-a-phase-vocoder-in-matlab.html>

A. MATLAB 代码实现

本章节提供了论文中使用的所有 MATLAB 源代码。代码按功能模块组织，包括主程序、核心算法、分析工具和测试演示。

A.1 主程序文件

```

1 function main_voice_processing()
2 % 语音信号处理系统
3
4 clc; clear; close all;
5 addpath("io"); addpath("analysis"); addpath("core"); addpath("effects")
   ; addpath("filters"); addpath("utils");
6
7 fprintf('== 语音信号处理系统 ==\n');
8 fprintf('1. 录音与读取\n');
9 fprintf('2. 频谱分析\n');
10 fprintf('3. 滤波器设计与应用\n');
11 fprintf('4. 变声处理\n');
12 fprintf('5. 时间拉伸\n');
13 fprintf('6. 结果播放与保存\n\n');
14
15 fs = 44100; % 采样率
16 duration = 5; % 录音时长
17
18 try
19     %% 1. 录音或读取音频文件
20     choice = input('选择输入方式 (1-录音, 2-读取文件): ');
21
22     if choice == 1
23         fprintf('开始录音 %d 秒...\n', duration);
24         x = record_audio(duration, fs);
25         filename = 'recorded_audio.wav';
26         audiowrite(filename, x, fs);
27         fprintf('录音已保存为: %s\n', filename);
28     else
29         [filename, pathname] = uigetfile({'*.wav;*.mp3;*.m4a', '音频文件
   '});
30         if filename == 0
31             error('未选择文件');
32         end
33         filepath = fullfile(pathname, filename);
34         [x, fs] = read_audio(filepath);
35         fprintf('读取文件: %s, 采样率: %d Hz, 时长: %.2f 秒\n', ...
36                 filename, fs, length(x)/fs);
37     end

```

```

38
39 % 转换为单声道
40 if size(x, 2) > 1
41     x = mean(x, 2);
42 end
43
44 %% 2. 频谱分析
45 fprintf('\n=== 频谱分析 ===\n');
46 spectrum_analysis(x, fs, '原始信号分析');
47
48 %% 3. 滤波器设计与应用
49 fprintf('\n=== 滤波器设计 ===\n');
50
51 fc_low = 100; % 低频截止
52 fc_high = 3500; % 高频截止
53 filter_order = 4;
54
55 try
56     [b, a] = design_filter('bandpass', fc_low, fc_high, fs,
57                             filter_order);
58     plot_frequency_response(b, a, fs, '带通滤波器频响');
59
60     x_filtered = apply_filter(x, b, a);
61     spectrum_analysis(x_filtered, fs, '滤波后信号分析');
62
63 catch ME
64     fprintf('滤波器处理失败: %s\n', ME.message);
65     fprintf('使用原始信号继续处理\n');
66     x_filtered = x;
67 end
68
69 %% 4. 变声处理
70 fprintf('\n=== 变声处理 ===\n');
71 semitones = input('输入变调半音数 (正数升调, 负数降调): ');
72
73 y_pitch = pitch_shift(x_filtered, semitones, fs, ...
74                        'formant_correction', true, 'nfft', 1024,
75                        'method', 'cepstral', 'beta', 1.15);
76 spectrum_analysis(y_pitch, fs, '变调后信号分析');
77
78 %% 5. 时间拉伸
79 fprintf('\n=== 时间拉伸 ===\n');
80 time_ratio = input('输入时间拉伸比例 (>1变慢, <1变快): ');
81
82 y_time = time_stretch(x_filtered, time_ratio, fs);

```

```

83     spectrum_analysis(y_time, fs, '变速后信号分析');
84
85
86     %% 6. 结果显示与播放
87     fprintf('\n=== 结果显示 ===\n');
88
89     plot_signals({x, x_filtered, y_pitch, y_time}, fs, ...
90                 {'原始信号', '滤波后信号', '变调信号', '时间拉伸信号'});
91
92     % 播放结果对比
93     play_comparison(x, y_pitch, y_time, fs);
94
95     % 保存结果
96     output_file = 'processed_audio.wav';
97     audiowrite(output_file, y_pitch, fs);
98     fprintf('处理结果已保存为: %s\n', output_file);
99
100    fprintf('\n=== 处理完成 ===\n');
101
102    catch ME
103        fprintf('错误: %s\n', ME.message);
104    end
105 end
106
107 function play_comparison(original, pitch, time, fs)
108 % 播放原始和处理后的音频对比
109     fprintf('播放原始音频...\n');
110     sound(original, fs);
111     pause(length(original)/fs + 1);
112
113     sound(pitch, fs);
114     pause(length(pitch)/fs + 1);
115
116     fprintf('播放处理后的音频...\n');
117     sound(time, fs);
118 end

```

Listing 1: 语音信号处理系统主程序 - 集成录音、频谱分析、滤波、变声和时间拉伸功能

A.2 核心算法实现

A.2.1 相位声码器时间拉伸

```

1 function y = pvoc(x, r, n)
2 % 相位声码器时间拉伸
3 % x: 输入信号, r: 时间比例因子, n: FFT大小
4

```

```

5  if nargin < 3
6      n = 1024;
7  end
8
9  % 25%重叠的STFT
10 hop = n/4;
11 scf = 1.0;
12
13 X = scf * my_stft(x', n, n, hop);
14
15 [rows, cols] = size(X);
16 t = 0:r:(cols-2);
17
18 X2 = pvsample(X, t, hop);
19
20 % 逆变换回波形
21 y = my_istft(X2, n, n, hop)';
22
23 % ... 省略相位计算和插值细节

```

Listing 2: 相位声码器时间拉伸算法实现

A.2.2 STFT 频谱插值

```

1  function c = pvsample(b, t, hop)
2  % STFT数组插值 - 相位声码器核心
3  % b: STFT数组, t: 时间采样点, hop: 帧移
4
5  [rows, cols] = size(b);
6  N = 2*(rows-1);
7
8  if hop == 0
9      hop = N/2;
10 end
11
12 c = zeros(rows, length(t));
13
14 % 计算每个频段的相位增量
15 dphi = zeros(1, N/2+1);
16 dphi(2:(1 + N/2)) = (2*pi*hop) ./ (N ./ (1:(N/2)));
17
18 ph = angle(b(:, 1));
19
20 b = [b, zeros(rows, 1)];
21
22 ocol = 1;
23 for tt = t

```

```

24 % 获取两个STFT列
25 bcols = b(:, floor(tt)+[1 2]);
26 tf = tt - floor(tt);
27
28 % 幅度插值
29 bmag = (1-tf)*abs(bcols(:,1)) + tf*(abs(bcols(:,2)));
30
31 % 计算相位差
32 dp = angle(bcols(:,2)) - angle(bcols(:,1)) - dphi';
33 dp = dp - 2 * pi * round(dp/(2*pi));
34
35 % 保存处理后的列
36 c(:,ocol) = bmag .* exp(j*ph);
37
38 % 累积相位
39 ph = ph + dphi' + dp;
40 ocol = ocol+1;
41 end
42
43 % ... 省略相位展开和边界处理细节

```

Listing 3: STFT 数组插值算法（相位声码器核心）

A.2.3 完整变调功能

```

1 function y = pitch_shift_complete(x, semitones, fs, varargin)
2 % 完整变调不变速函数
3 % x: 输入信号, semitones: 半音变化, fs: 采样率
4
5 p = inputParser;
6 addRequired(p, 'x', @isvector);
7 addRequired(p, 'semitones', @isnumeric);
8 addRequired(p, 'fs', @(x) x > 0);
9 addParameter(p, 'nfft', 1024, @(x) x > 0);
10 addParameter(p, 'formant_correction', true, @islogical);
11 addParameter(p, 'method', 'lpc', @ischar);
12 addParameter(p, 'beta', 1.5, @isnumeric);
13 parse(p, x, semitones, fs, varargin{:});
14
15 if semitones == 0
16     y = x;
17     return;
18 end
19
20 x = x(:);
21 pitch_ratio = 2^(semitones/12);
22

```

```

23 try
24     if semitones > 0
25         % 升调：先拉伸时间，再重采样升调
26         stretch_factor = 1 / pitch_ratio;
27         time_stretched = pvoc(x, stretch_factor, p.Results.nfft);
28         y = resample_pitch(time_stretched, pitch_ratio, fs, 'quality', '
           high');
29     else
30         % 降调：先压缩时间，再重采样降调
31         stretch_factor = abs(1 / pitch_ratio);
32         time_compressed = pvoc(x, stretch_factor, p.Results.nfft);
33         y = resample_pitch(time_compressed, pitch_ratio, fs, 'quality', '
           high');
34     end
35
36     % 共振峰校正
37     if p.Results.formant_correction && abs(semitones) >= 2
38         y = formant_correction(x, y, semitones, fs, ...
39             'intensity', 0.6, 'method', p.Results.method, 'beta', p.Results
              .beta);
40     end
41
42     % 长度和幅度调整
43     min_len = min(length(x), length(y));
44     y = y(1:min_len);
45     y = y / max(abs(y)) * max(abs(x));
46
47 catch ME
48     error('变调处理失败：%s', ME.message);
49 end
50
51 end

```

Listing 4: 变调不变速完整实现

A.2.4 共振峰校正方法

```

1 function y = formant_correction_main(x, y, semitones, fs, varargin)
2 % 共振峰校正 - 提高变调后的音质自然度
3 % x: 原信号, y: 变调后的信号, semitones: 半音变化量, fs: 采样率
4
5 p = inputParser;
6 addRequired(p, 'x', @isvector);
7 addRequired(p, 'semitones', @isnumeric);
8 addRequired(p, 'fs', @(x) x > 0);
9 addParameter(p, 'intensity', 0.5, @(x) x >= 0 && x <= 1);
10 addParameter(p, 'method', 'cepstral', @ischar);

```

```

11 addParameter(p, 'beta', 1.5, @isnumeric);
12 parse(p, x, semitones, fs, varargin{:});
13
14 if semitones == 0
15     y = x;
16     return;
17 end
18
19 % 计算共振峰补偿因子
20 formant_compensation = 1.0 - (p.Results.intensity * abs(semitones) / 24);
21 formant_compensation = max(0.5, min(1.5, formant_compensation));
22
23 switch p.Results.method
24     case 'lpc'
25         y = formant_correction_lpc(y, formant_compensation, fs);
26     case 'cepstral'
27         cepstral_order = round(fs / 1000);
28         y = formant_preservation_cepstral(x, y, fs, cepstral_order, p.
            Results.beta);
29     otherwise
30         y = formant_correction_lpc(y, formant_compensation, fs);
31 end
32
33 end
34
35 function y = formant_correction_lpc(x, compensation, fs)
36 % 基于LPC的共振峰校正
37 frame_size = round(0.03 * fs);
38 hop_size = round(frame_size / 4);
39
40 frames = buffer(x, frame_size, frame_size - hop_size, 'nodelay');
41 y_frames = zeros(size(frames));
42
43 for i = 1:size(frames, 2)
44     frame = frames(:, i);
45
46     if max(abs(frame)) > 1e-6
47         % LPC分析
48         lpc_order = min(12, round(fs/1000) + 2);
49         a = lpc(frame, lpc_order);
50
51         % 调整共振峰频率
52         roots_a = roots(a);
53         angles = angle(roots_a);
54         magnitudes = abs(roots_a);
55
56         % 应用补偿

```

```

57     new_angles = angles * compensation;
58     new_roots = magnitudes .* exp(1j * new_angles);
59
60     % 重建LPC系数
61     new_a = real(poly(new_roots));
62
63     % LPC合成
64     residual = filter(a, 1, frame);
65     y_frame = filter(1, new_a, residual);
66
67     y_frames(:, i) = y_frame;
68 end
69 end
70
71 % 重叠相加合成
72 y = overlap_add(y_frames, hop_size);
73 y = y(1:length(x));
74
75 end
76
77 function y = formant_preservation_cepstral(original_signal,
78     pitch_shifted_signal, fs, cepstral_order, beta)
79 % 基于倒谱域的共振峰保持算法
80 % ... 省略详细的迭代估计和频域处理代码
81
82 frame_size = round(0.03 * fs);
83 hop_size = round(frame_size / 4);
84 nfft = 2^nextpow2(frame_size * 2);
85
86 % 确保信号长度一致
87 min_len = min(length(original_signal), length(pitch_shifted_signal));
88 original_signal = original_signal(1:min_len);
89 pitch_shifted_signal = pitch_shifted_signal(1:min_len);
90
91 % 分帧处理
92 original_frames = buffer(original_signal, frame_size, frame_size - hop_size
93     , 'nodelay');
94 shifted_frames = buffer(pitch_shifted_signal, frame_size, frame_size -
95     hop_size, 'nodelay');
96
97 num_frames = size(original_frames, 2);
98 y_frames = zeros(frame_size, num_frames);
99
100 for i = 1:num_frames
    orig_frame = original_frames(:, i) .* window;

```

```

101 shifted_frame = shifted_frames(:, i) .* window;
102
103 if max(abs(orig_frame)) > 1e-6 && max(abs(shifted_frame)) > 1e-6
104     % 计算频谱
105     X = fft(orig_frame, nfft);
106     Y = fft(shifted_frame, nfft);
107
108     % 估计频谱包络
109     env_X = estimate_spectral_envelope(log(abs(X) + 1e-10),
110         cepstral_order);
111     env_Y = estimate_spectral_envelope(log(abs(Y) + 1e-10),
112         cepstral_order);
113
114     % 频率轴缩放
115     env_target = scale_spectral_envelope(env_X, beta);
116
117     % 计算包络比率
118     envelope_ratio = exp(env_target - env_Y);
119
120     % 应用校正
121     Y_corrected = Y .* envelope_ratio;
122
123     % 逆变换回时域
124     y_frame = real(ifft(Y_corrected, nfft));
125     y_frame = y_frame(1:frame_size) .* window;
126
127     y_frames(:, i) = y_frame;
128 else
129     y_frames(:, i) = shifted_frame;
130 end
131 end
132
133 % 重叠相加合成
134 y = overlap_add_synthesis(y_frames, hop_size, min_len);
135 y = normalize_ola_gain(y, window, hop_size, min_len);
136
137 end
138
139 function env = estimate_spectral_envelope(log_spectrum, cepstral_order)
140 % 估计频谱包络（倒谱低通滤波）
141 nfft = length(log_spectrum);
142 cepstrum = real(ifft(log_spectrum));
143
144 % 创建低通lifter
145 lifter = zeros(nfft, 1);
146 lifter(1:cepstral_order+1) = 1;
147 lifter(nfft - cepstral_order + 1:nfft) = 1;

```

```

146
147 % 应用 lifter
148 filtered_cepstrum = cepstrum .* lifter;
149 env = real(fft(filtered_cepstrum));
150
151 end
152
153 function env_scaled = scale_spectral_envelope(env, beta)
154 % 频谱包络频率轴缩放
155 n = length(env);
156 freq = linspace(0, 1, n).';
157 freq_scaled = freq / beta;
158 freq_scaled(freq_scaled > 1) = 1;
159 freq_scaled(freq_scaled < 0) = 0;
160
161 env_scaled = interp1(freq, env, freq_scaled, 'linear', 'extrap');
162
163 end
164
165 function y = overlap_add(frames, hop_size)
166 % 重叠相加合成
167 [frame_size, num_frames] = size(frames);
168 y_length = (num_frames - 1) * hop_size + frame_size;
169 y = zeros(y_length, 1);
170
171 for i = 1:num_frames
172     start_idx = (i - 1) * hop_size + 1;
173     end_idx = start_idx + frame_size - 1;
174     y(start_idx:end_idx) = y(start_idx:end_idx) + frames(:, i);
175 end
176
177 end
178
179 function y = overlap_add_synthesis(frames, hop_size, output_length)
180 % 重叠相加合成 (带长度控制)
181 y = overlap_add(frames, hop_size);
182 if length(y) > output_length
183     y = y(1:output_length);
184 end
185 end
186
187 function y = normalize_ola_gain(signal, window, hop_size, output_length)
188 % 归一化重叠相加增益
189 frame_size = length(window);
190 num_frames = ceil((output_length - frame_size) / hop_size) + 1;
191 gain_length = (num_frames - 1) * hop_size + frame_size;
192 gain = zeros(gain_length, 1);

```

```

193
194 window_squared = window .^ 2;
195
196 for i = 1:num_frames
197     start_idx = (i - 1) * hop_size + 1;
198     end_idx = start_idx + frame_size - 1;
199     gain(start_idx:end_idx) = gain(start_idx:end_idx) + window_squared;
200 end
201
202 gain = gain(1:output_length);
203 gain(gain < 1e-6) = 1;
204
205 y = signal ./ gain;
206
207 end

```

Listing 5: 共振峰校正主函数（支持 LPC 和倒谱域方法）

A.3 分析与可视化工具

A.3.1 共振峰对比分析

```

1 function plot_formant_comparison_demo(x, fs, semitones_up, varargin)
2 % 共振峰校正效果对比绘图（简化版）
3 % 输入: x - 原始信号, fs - 采样率, semitones_up - 变调半音数
4
5 addpath("core")
6
7 % 参数解析
8 p = inputParser;
9 addRequired(p, 'x', @isnumeric);
10 addRequired(p, 'fs', @isscalar);
11 addRequired(p, 'semitones_up', @isscalar);
12 addParameter(p, 'methods', {'lpc', 'cepstral'}, @iscell);
13 addParameter(p, 'show_plots', true, @islogical);
14 addParameter(p, 'save_figures', false, @islogical);
15 parse(p, x, fs, semitones_up, varargin{:});
16
17 methods = p.Results.methods;
18 show_plots = p.Results.show_plots;
19
20 fprintf('== 共振峰校正效果对比 ==\n');
21 fprintf('信号长度: %d 样本 (%.2f 秒)\n', length(x), length(x)/fs);
22 fprintf('采样率: %d Hz\n', fs);
23 fprintf('变调量: %+d 半音\n', semitones_up);
24 fprintf('校正方法: %s\n', strjoin(methods, ', '));
25

```

```

26 % 1. 核心处理
27 y_ps = pitch_shift(x, semitones_up, fs, 'formant_correction', false);
28 y_fc = pitch_shift(x, semitones_up, fs, 'formant_correction', true);
29
30 % 确保信号长度一致
31 min_len = min([length(x), length(y_ps), length(y_fc)]);
32 x = x(1:min_len);
33 y_ps = y_ps(1:min_len);
34 y_fc = y_fc(1:min_len);
35
36 % 应用不同的共振峰校正方法
37 results = struct();
38 for i = 1:length(methods)
39     method = methods{i};
40     fprintf('正在处理 %s 方法...\n', method);
41
42     switch method
43     case 'lpc'
44         y = pitch_shift(x, semitones_up, fs, 'formant_correction', true, 'method', 'lpc');
45     case 'cepstral'
46         y = pitch_shift(x, semitones_up, fs, 'formant_correction', true, 'method', 'cepstral');
47     otherwise
48         error('未知的共振峰校正方法: %s', method);
49     end
50
51     results(i).method = method;
52     results(i).signal = y(1:min_len);
53 end
54
55 % 2. 生成对比图表
56 if show_plots
57     plot_time_domain_simple(x, y_ps, y_fc, results, fs, semitones_up);
58     plot_spectrum_simple(x, y_ps, y_fc, results, fs, semitones_up);
59 end
60
61 fprintf('\n=== 对比分析完成 ===\n');
62 end
63
64 function plot_time_domain_simple(x, y_ps, y_fc, results, fs, semitones_up)
65 % 简化时域波形对比
66 fprintf('生成时域波形对比图...\n');
67
68 figure('Name', sprintf('时域波形对比 (变调: %d 半音)', semitones_up), ...
69     'Position', [100, 100, 1000, 600], 'NumberTitle', 'off');
70

```

```

71 t = (0:length(x)-1)/fs;
72 t_ps = (0:length(y_ps)-1)/fs;
73 t_fc = (0:length(y_fc)-1)/fs;
74
75 % 原始信号
76 subplot(2,2,1);
77 plot(t, x, 'k', 'LineWidth', 1.5);
78 title('原始信号', 'FontSize', 12, 'FontWeight', 'bold');
79 grid on; axis tight;
80
81 % 变调后对比
82 subplot(2,2,2);
83 plot(t, x, 'k', 'LineWidth', 1.5); hold on;
84 plot(t_ps, y_ps, 'r', 'LineWidth', 1.5);
85 title('变调后对比', 'FontSize', 12, 'FontWeight', 'bold');
86 legend({'原始', '变调后'}, 'Location', 'best');
87 grid on;
88
89 % 校正后对比
90 subplot(2,2,3);
91 plot(t, x, 'k', 'LineWidth', 1.5); hold on;
92 plot(t_fc, y_fc, 'b', 'LineWidth', 1.5);
93 title('校正后对比', 'FontSize', 12, 'FontWeight', 'bold');
94 legend({'原始', '校正后'}, 'Location', 'best');
95 grid on;
96
97 % 各方法结果
98 subplot(2,2,4);
99 colors = {'g', 'm'};
100 plot(t, x, 'k', 'LineWidth', 1.5); hold on;
101 for i = 1:length(results)
102     y = results(i).signal;
103     t_y = (0:length(y)-1)/fs;
104     plot(t_y, y, colors{i}, 'LineWidth', 1.2, 'DisplayName', upper(results(
105         i).method));
106 end
107 title('各方法对比', 'FontSize', 12, 'FontWeight', 'bold');
108 legend('show', 'Location', 'best');
109 grid on;
110 sgtitle(sprintf('时域波形对比 (Pitch Shift: %d semitones)', semitones_up),
111     ...
112     'FontSize', 14, 'FontWeight', 'bold');
113 end
114 function plot_spectrum_simple(x, y_ps, y_fc, results, fs, semitones_up)
115 % 简化频谱对比

```

```

116 fprintf(' 生成频谱对比图...\n');
117
118 figure('Name', sprintf('频谱对比 (变调: %d 半音)', semitones_up), ...
119       'Position', [150, 100, 800, 600], 'NumberTitle', 'off');
120
121 % 计算频谱
122 [f_orig, mag_orig] = calculate_spectrum_simple(x, fs);
123 [f_ps, mag_ps] = calculate_spectrum_simple(y_ps, fs);
124 [f_fc, mag_fc] = calculate_spectrum_simple(y_fc, fs);
125
126 % 原始信号频谱
127 subplot(2,2,1);
128 plot(f_orig, mag_orig, 'k', 'LineWidth', 1.5);
129 title('原始信号频谱', 'FontSize', 12, 'FontWeight', 'bold');
130 grid on; xlim([0 4000]); ylim([-80, 5]);
131
132 % 变调后频谱对比
133 subplot(2,2,2);
134 plot(f_orig, mag_orig, 'k', 'LineWidth', 1.5); hold on;
135 plot(f_ps, mag_ps, 'r', 'LineWidth', 1.5);
136 title('变调后频谱对比', 'FontSize', 12, 'FontWeight', 'bold');
137 legend({'原始', '变调后'}, 'Location', 'best');
138 grid on; xlim([0 4000]); ylim([-80, 5]);
139
140 % 校正后频谱对比
141 subplot(2,2,3);
142 plot(f_orig, mag_orig, 'k', 'LineWidth', 1.5); hold on;
143 plot(f_fc, mag_fc, 'b', 'LineWidth', 1.5);
144 title('校正后频谱对比', 'FontSize', 12, 'FontWeight', 'bold');
145 legend({'原始', '校正后'}, 'Location', 'best');
146 grid on; xlim([0 4000]); ylim([-80, 5]);
147
148 % 各方法频谱对比
149 subplot(2,2,4);
150 colors = {'g', 'm'};
151 plot(f_orig, mag_orig, 'k', 'LineWidth', 1.5); hold on;
152 for i = 1:length(results)
153     [f_result, mag_result] = calculate_spectrum_simple(results(i).signal,
154                                                         fs);
155     plot(f_result, mag_result, colors{i}, 'LineWidth', 1.2, ...
156          'DisplayName', upper(results(i).method));
157 end
158 title('各方法频谱对比', 'FontSize', 12, 'FontWeight', 'bold');
159 legend('show', 'Location', 'best');
160 grid on; xlim([0 4000]); ylim([-80, 5]);
161 sgtitle(sprintf('频谱对比 (Pitch Shift: %d semitones)', semitones_up), ...

```

```

162     'FontSize', 14, 'FontWeight', 'bold');
163 end
164
165 function [f, mag] = calculate_spectrum_simple(x, fs)
166 % 简化频谱计算
167 N = length(x);
168 if N < 256
169     f = 0:100:4000;
170     mag = zeros(size(f)) - 80;
171     return;
172 end
173
174 X = fft(x);
175 f = (0:floor(N/2)) * fs / N;
176 mag = 20*log10(abs(X(1:floor(N/2)+1)) + eps);
177
178 if ~isempty(mag) && max(mag) > min(mag)
179     mag = mag - max(mag);
180 end
181 end

```

Listing 6: 共振峰校正效果对比可视化

A.3.2 LPC 共振峰估计

```

1 function [F, B, A] = formant_lpc(x, fs, order, winlen, preemph)
2 % LPC 共振峰估计 (简化版)
3
4 if nargin < 3, order = 12; end
5 if nargin < 4, winlen = 0.025; end
6 if nargin < 5, preemph = 0.63; end
7
8 % 1. 预处理
9 x = x(:); % 转为列向量
10 if preemph > 0
11     x = filter([1 -preemph], 1, x);
12 end
13
14 % 2. 分帧加窗
15 nwin = round(winlen * fs);
16 w = hamming(nwin);
17 if length(x) < nwin
18     x = [x; zeros(nwin - length(x), 1)];
19 end
20
21 % 3. LPC 分析
22 [a, e] = lpc(x, order);

```

```

23
24 % 4. 求根
25 rts = roots(a);
26 rts = rts(imag(rts) > 0); % 只保留上半平面
27
28 % 5. 计算频率和带宽
29 ang = angle(rts);
30 freqs = ang * fs / (2 * pi);
31 bws = -0.5 * fs * log(abs(rts)) / (2 * pi);
32
33 % 6. 筛选有效共振峰
34 valid_idx = (freqs > 90) & (freqs < 4000) & (bws < 700);
35 F = freqs(valid_idx);
36 B = bws(valid_idx);
37
38 % 7. 排序
39 [sorted_freqs, idx] = sort(F);
40 F = sorted_freqs;
41 B = B(idx);
42
43 % 8. 返回 LPC 系数
44 A = a;
45
46 end

```

Listing 7: 基于 LPC 的共振峰频率估计函数

A.4 测试与演示程序

A.4.1 变调共振峰分析演示

```

1 function test_pitch_shift_formants_demo()
2 % 变调共振峰分析演示（简化版）
3
4 addpath('core')
5
6 % 1. 加载数据
7 S = load('mtlb.mat');
8 mtlb = S.mtlb;
9 Fs = S.Fs;
10 dt = 1/Fs;
11 I0 = round(0.1/dt);
12 Iend = round(0.25/dt);
13 x = mtlb(I0:Iend);
14
15 % 2. 变调处理（升高 4 个半音）
16 shift_semitones = -3;

```

```

17 disp(['正在执行变调 (升调 ', num2str(shift_semitones), ' 半音)...']);
18
19 % 调用变调函数
20 mtlb_shifted = pitch_shift(mtlb, shift_semitones, Fs, 'formant_correction',
    false);
21 mtlb_shifted_lpc = pitch_shift(mtlb, shift_semitones, Fs, '
    formant_correction', true, 'method', 'lpc');
22 mtlb_shifted_cepstral = pitch_shift(mtlb, shift_semitones, Fs, '
    formant_correction', true, 'method', 'cepstral');
23
24 % 对齐长度
25 len = min([length(mtlb), length(mtlb_shifted_lpc), length(
    mtlb_shifted_cepstral)]);
26 mtlb = mtlb(1:len);
27 mtlb_shifted = mtlb_shifted(1:len);
28 mtlb_shifted_lpc = mtlb_shifted_lpc(1:len);
29 mtlb_shifted_cepstral = mtlb_shifted_cepstral(1:len);
30
31 % 3. 提取共振峰
32 params.lpcOrder = 8;
33 params.winLen = 0.030;
34 params.timeStep = 0.010;
35 params.preemph = [1 0.63];
36
37 disp('提取原语音共振峰...');
38 [t_orig, f_orig] = extract_formants_doc_style(mtlb, Fs, params);
39
40 disp('提取变调语音共振峰...');
41 [t_shift, f_shift] = extract_formants_doc_style(mtlb_shifted, Fs, params);
42
43 disp('提取 LPC 变调语音共振峰...');
44 [t_shift_lpc, f_shift_lpc] = extract_formants_doc_style(mtlb_shifted_lpc,
    Fs, params);
45
46 disp('提取倒谱域 变调语音共振峰...');
47 [t_shift_cepstral, f_shift_cepstral] = extract_formants_doc_style(
    mtlb_shifted_cepstral, Fs, params);
48
49 % 4. 绘制对比图
50 signals_to_plot = {'original', 'lpc', 'cepstral'};
51 time_axis = (0:len-1)/Fs;
52
53 % 调用绘图函数
54 plot_formant_comparison_simple(time_axis, mtlb, mtlb_shifted,
    mtlb_shifted_lpc, mtlb_shifted_cepstral, ...
55     t_orig, f_orig, t_shift, f_shift, t_shift_lpc, f_shift_lpc,
    t_shift_cepstral, f_shift_cepstral, ...

```

```

56     signals_to_plot, Fs);
57
58 disp('分析完成。');
59
60 % 5. 量化评估
61 try
62     % 数据对齐
63     common_time = t_orig(t_orig <= t_shift(end));
64     F1_orig_aligned = f_orig(1:length(common_time), 1);
65     F2_orig_aligned = f_orig(1:length(common_time), 2);
66
67     F1_shift_aligned = interp1(t_shift, f_shift_cepstral(:,1), common_time,
        'linear');
68     F2_shift_aligned = interp1(t_shift, f_shift_cepstral(:,2), common_time,
        'linear');
69
70     % 计算指标
71     pitch_ratio = 2^(shift_semitones/12);
72     ratio_F1 = mean(F1_shift_aligned ./ F1_orig_aligned, 'omitnan');
73     ratio_F2 = mean(F2_shift_aligned ./ F2_orig_aligned, 'omitnan');
74     diff_F1 = mean(F1_shift_aligned - F1_orig_aligned, 'omitnan');
75     diff_F2 = mean(F2_shift_aligned - F2_orig_aligned, 'omitnan');
76
77     % 打印结果
78     fprintf('\n===== 变调共振峰评估报告 =====\n');
79     fprintf('变调半音数: %d\n', shift_semitones);
80     fprintf('理论变调倍率: %.4f\n', pitch_ratio);
81     fprintf('F1 平均实测倍率: %.4f (偏差: %.2f%%)\n', ratio_F1, abs(
        ratio_F1 - pitch_ratio)/pitch_ratio*100);
82     fprintf('F2 平均实测倍率: %.4f (偏差: %.2f%%)\n', ratio_F2, abs(
        ratio_F2 - pitch_ratio)/pitch_ratio*100);
83     fprintf('F1 平均频率移动: %.2f Hz\n', diff_F1);
84     fprintf('F2 平均频率移动: %.2f Hz\n', diff_F2);
85     fprintf('===== \n');
86
87     % 结论
88     if abs(ratio_F1 - 1) < 0.05
89         disp('结论: 共振峰基本未移动, 音色保持良好。');
90     elseif abs(ratio_F1 - pitch_ratio) < 0.05
91         disp('结论: 共振峰随音高成比例移动, 存在"小黄人效应"。');
92     else
93         disp('结论: 共振峰变化不规律。');
94     end
95
96 catch ME
97     warning('量化评估失败: %s', ME.message);
98 end

```

```

99
100 end
101
102 function plot_formant_comparison_simple(time_axis, mtlb, mtlb_shifted,
    mtlb_shifted_lpc, mtlb_shifted_cepstral, ...
103     t_orig, f_orig, t_shift, f_shift, t_shift_lpc, f_shift_lpc,
    t_shift_cepstral, f_shift_cepstral, ...
104     signals_to_plot, fs)
105 % 简化共振峰对比绘图
106
107 signal_configs = struct(...
108     'original', struct('name', '原始', 'color', 'k', 'data', mtlb, 'time',
    t_orig, 'formants', f_orig), ...
109     'shifted', struct('name', '变调', 'color', [1 0 0], 'data',
    mtlb_shifted, 'time', t_shift, 'formants', f_shift), ...
110     'lpc', struct('name', 'LPC校正', 'color', [0 0 1], 'data',
    mtlb_shifted_lpc, 'time', t_shift_lpc, 'formants', f_shift_lpc), ...
111     'cepstral', struct('name', '倒谱校正', 'color', [0 1 0], 'data',
    mtlb_shifted_cepstral, 'time', t_shift_cepstral, 'formants',
    f_shift_cepstral) ...
112 );
113
114 figure('Color', 'w', 'Position', [100, 100, 1000, 600]);
115
116 % 波形对比
117 subplot(3,1,1);
118 for i = 1:length(signals_to_plot)
119     sig_name = signals_to_plot{i};
120     if isfield(signal_configs, sig_name)
121         config = signal_configs.(sig_name);
122         plot(time_axis, config.data, 'Color', config.color, 'DisplayName',
    config.name);
123         hold on;
124     end
125 end
126 title('波形对比');
127 legend('Location', 'best');
128 grid on; axis tight;
129
130 % F1 共振峰对比
131 subplot(3,1,2);
132 plot_formant_tracks_simple(signals_to_plot, signal_configs, 1, 'F1 共振峰对
    比', fs);
133
134 % F2 共振峰对比
135 subplot(3,1,3);
136 plot_formant_tracks_simple(signals_to_plot, signal_configs, 2, 'F2 共振峰对

```

```

        比', fs);
137
138 end
139
140 function plot_formant_tracks_simple(signals_to_plot, signal_configs,
        formant_idx, title_str, fs)
141 % 简化共振峰轨迹绘制
142
143 for i = 1:length(signals_to_plot)
144     sig_name = signals_to_plot{i};
145     if isfield(signal_configs, sig_name)
146         config = signal_configs.(sig_name);
147         if ischar(config.color)
148             color_val = config.color;
149         else
150             color_val = config.color(1:3);
151         end
152         plot(config.time, config.formants(:,formant_idx), '.', ...
153             'Color', color_val, 'MarkerSize', 6);
154         hold on;
155     end
156 end
157
158 legend_names = {};
159 for i = 1:length(signals_to_plot)
160     sig_name = signals_to_plot{i};
161     if isfield(signal_configs, sig_name)
162         config = signal_configs.(sig_name);
163         legend_names{end+1} = [config.name ' F' num2str(formant_idx)];
164     end
165 end
166 legend(legend_names, 'Location', 'best');
167 title(title_str);
168 ylabel('频率 (Hz)');
169 if formant_idx == 2
170     xlabel('时间 (s)');
171 end
172 grid on;
173 xlim([0, signal_configs.original.time(end)]);
174
175 end
176
177 function [t, f_tracks, b_tracks] = extract_formants_doc_style(x, fs, p)
178 % 提取共振峰 (简化版)
179 N = length(x);
180 nWin = round(p.winLen * fs);
181 nStep = round(p.timeStep * fs);

```

```

182 nFrames = floor((N - nWin) / nStep) + 1;
183
184 f_tracks = nan(nFrames, 3);
185 b_tracks = nan(nFrames, 3);
186 t = ((0:nFrames-1) * nStep + nWin/2) / fs;
187
188 w = hamming(nWin);
189
190 for i = 1:nFrames
191     idx_start = (i-1)*nStep + 1;
192     segment = x(idx_start : idx_start + nWin - 1);
193
194     if mean(segment.^2) < 1e-5
195         continue;
196     end
197
198     % 加窗和预加重
199     x1 = segment .* w;
200     x1 = filter(1, p.preemph, x1);
201
202     % LPC分析
203     [A, E] = lpc(x1, p.lpcOrder);
204     if any(isnan(A))
205         continue;
206     end
207
208     % 求根和频率转换
209     rts = roots(A);
210     rts = rts(imag(rts) >= 0);
211     angz = atan2(imag(rts), real(rts));
212     [frqs, indices] = sort(angz .* (fs / (2*pi)));
213     bw = -1/2 * (fs / (2*pi)) * log(abs(rts(indices)));
214
215     % 筛选共振峰
216     candidates_f = [];
217     candidates_b = [];
218
219     for k = 1:length(frqs)
220         if (frqs(k) > 90 && bw(k) < 400)
221             candidates_f = [candidates_f; frqs(k)];
222             candidates_b = [candidates_b; bw(k)];
223         end
224     end
225
226     % 保存前三个结果
227     num_found = length(candidates_f);
228     if num_found > 0

```

```

229         f_tracks(i, 1:min(3, num_found)) = candidates_f(1:min(3, num_found)
230             );
231         b_tracks(i, 1:min(3, num_found)) = candidates_b(1:min(3, num_found)
232             );
233     end
234 end

```

Listing 8: 变调过程中的共振峰特性分析演示

A.4.2 倒谱域校正演示

```

1 function test_cepstral_correction_demo()
2 % 倒谱域共振峰校正演示（简化版）
3
4 addpath('core')
5
6 % 1. 加载数据
7 S = load('mtlb.mat');
8 mtlb = S.mtlb;
9 Fs = S.Fs;
10
11 % 2. 参数设置
12 shift_semitones = -3; % 降低 3 个半音
13
14 % 3. 变调处理
15 disp('正在执行变调处理...');
16 mtlb_shifted = pitch_shift(mtlb, shift_semitones, Fs, 'formant_correction',
17     false);
18 mtlb_corrected = pitch_shift(mtlb, shift_semitones, Fs, 'formant_correction',
19     true, 'method', 'cepstral');
20
21 % 4. 对齐长度
22 len = min([length(mtlb), length(mtlb_shifted), length(mtlb_corrected)]);
23 mtlb = mtlb(1:len);
24 mtlb_shifted = mtlb_shifted(1:len);
25 mtlb_corrected = mtlb_corrected(1:len);
26
27 % 5. 绘制对比图
28 time_axis = (0:len-1)/Fs;
29
30 figure('Color', 'w', 'Position', [100, 100, 800, 600]);
31
32 % 波形对比
33 subplot(3,1,1);
34 plot(time_axis, mtlb, 'k', 'DisplayName', '原始');

```

```

33 hold on;
34 plot(time_axis, mtlb_shifted, 'r', 'DisplayName', '变调');
35 plot(time_axis, mtlb_corrected, 'g', 'DisplayName', '倒谱校正');
36 title('波形对比');
37 legend('Location', 'best');
38 grid on;
39
40 % 频谱对比 (取中间段)
41 mid_start = round(len/3);
42 mid_end = round(2*len/3);
43
44 % 原始信号频谱
45 subplot(3,1,2);
46 [P1, f1] = pwelch(mtlb(mid_start:mid_end), [], [], [], Fs);
47 semilogx(f1, 10*log10(P1), 'k', 'DisplayName', '原始');
48 hold on;
49 [P2, f2] = pwelch(mtlb_shifted(mid_start:mid_end), [], [], [], Fs);
50 semilogx(f2, 10*log10(P2), 'r', 'DisplayName', '变调');
51 [P3, f3] = pwelch(mtlb_corrected(mid_start:mid_end), [], [], [], Fs);
52 semilogx(f3, 10*log10(P3), 'g', 'DisplayName', '倒谱校正');
53 xlim([100 4000]);
54 title('频谱对比');
55 legend('Location', 'best');
56 grid on;
57
58 % 共振峰对比
59 subplot(3,1,3);
60 % 提取共振峰 (简化版)
61 [F_orig, ~] = formant_lpc(mtlb, Fs);
62 [F_shift, ~] = formant_lpc(mtlb_shifted, Fs);
63 [F_corr, ~] = formant_lpc(mtlb_corrected, Fs);
64
65 % 绘制前三个共振峰
66 bar([F_orig(1:3), F_shift(1:3), F_corr(1:3)]);
67 set(gca, 'XTickLabel', {'F1', 'F2', 'F3'});
68 legend('原始', '变调', '倒谱校正');
69 title('共振峰频率对比');
70 grid on;
71
72 % 6. 结果分析
73 disp('倒谱域共振峰校正演示完成。');
74 disp(['原始 F1: ', num2str(F_orig(1)) ' Hz']);
75 disp(['变调 F1: ', num2str(F_shift(1)) ' Hz']);
76 disp(['校正 F1: ', num2str(F_corr(1)) ' Hz']);
77
78 end

```

Listing 9: 倒谱域共振峰校正方法演示

B. 代码使用说明

本附录包含了语音信号处理系统的完整 MATLAB 实现。各模块功能如下：

- 主程序 (1)：系统入口，集成所有功能
- 相位声码器 (2)：实现时间尺度修改
- 变调算法 (4)：实现音高变换
- 共振峰校正 (5)：LPC 与倒谱域方法
- 分析工具：提供可视化与性能评估

所有代码均在 MATLAB R2024b 环境下测试通过，需安装 Audio Toolbox。